

Optimization

3.3 Derivative-based methods

José Firmino Aguilar Madeira

Why use derivatives

The methods of 3.2 only compare values of f and therefore need many evaluations (Rosenbrock: n_f from 166 to 13 445). The gradient gives **directly** the local direction of steepest ascent — and hence $-\nabla f$ that of steepest descent.

When to use them

f differentiable, derivatives available (analytical, by finite differences or by automatic differentiation) and **sufficiently reliable**. Then they can be much faster.

When not to use them

f discontinuous or nondifferentiable, or with noise that makes the derivatives unreliable: go back to 3.2.

Discrete variables: the usual gradient does not apply directly — another formulation or another method is needed.

In this section: the cost of obtaining derivatives in $\mathbb{R}^n$, what a descent direction is, and three methods: steepest descent (3.3.1), Newton (3.3.2), conjugate gradients (3.3.3).

Numerical derivatives in n dimensions

The central formulas of 2.6, coordinate by coordinate, with steps h_i :

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + h_i e_i) - f(x - h_i e_i)}{2 h_i}, \quad \frac{\partial^2 f}{\partial x_i^2} \approx \frac{f(x + h_i e_i) - 2f(x) + f(x - h_i e_i)}{h_i^2},$$

$$\frac{\partial^2 f}{\partial x_i \partial x_j} \approx \frac{f(x + h_i e_i + h_j e_j) - f(x + h_i e_i - h_j e_j) - f(x - h_i e_i + h_j e_j) + f(x - h_i e_i - h_j e_j)}{4 h_i h_j}$$

	evaluations of f
gradient (n partials)	$2n$
Hessian diagonal	$2n + 1^*$
off-diagonal ($n(n-1)/2$)	4 each
full Hessian	$2n^2 + 1$

* reuses $f(x \pm h_i e_i)$ if already computed for the gradient.

- For $n = 2$: gradient 4, Hessian 9. For $n = 10$: 20 and 201. For $n = 100$: 200 and 20 001.
- Same trade-off as in 2.6 when choosing h_i (truncation vs. round-off).
- Example: Rosenbrock at $(-1.5, 2)$, $\nabla f = (-155, -50)$; with $h = 10^{-4}$ the error is $6 \cdot 10^{-6}$.
- This is why first-order methods (3.3.1, 3.3.3) are attractive when n is large.

From 3.1: what do we already know about directions?

Quick review (3.1):

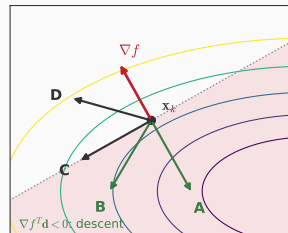
- $f(x + \alpha d) \approx f(x) + \alpha \nabla f(x)^T d$ for small $\alpha > 0$;
- d is a **descent direction** if $\nabla f(x)^T d < 0$ (angle with ∇f greater than 90°);
- for fixed $\|d\|$, $\nabla f^T d$ is minimal at $d = -\nabla f$: **steepest descent**.

What is new: a family of descent directions

If M is **positive definite** and $\nabla f(x) \neq 0$,
 $d = -M \nabla f(x) \Rightarrow \nabla f^T d = -\nabla f^T M \nabla f < 0$.

$M = I$: steepest descent (3.3.1). $M = H^{-1}$, $H = \nabla^2 f(x)$,
when H is positive definite: Newton (3.3.2).

The same idea unifies the two methods: changing M means changing the “metric” in which we descend.



The figure from 3.1: A and B descend; C is tangent; D ascends.

The scheme common to the three methods

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k, \quad \nabla f(\mathbf{x}_k)^T \mathbf{d}_k < 0$$

3.3 chooses $\mathbf{d}_k$

Ch. 2 chooses α_k

In descent methods — including Newton when H_k is positive definite or the direction is safeguarded (3.3.2) — we seek $\nabla f(\mathbf{x}_k)^T \mathbf{d}_k < 0$; pure Newton does not guarantee it if H_k is indefinite.

Method	Direction $\mathbf{d}_k$	Information
3.3.1 Steepest descent (Cauchy)	$-\nabla f(\mathbf{x}_k)$	first order
3.3.2 Newton	$-\nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$	second order
3.3.3 Conjugate gradients (Fletcher–Reeves)	$-\nabla f(\mathbf{x}_k) + \beta_k \mathbf{d}_{k-1}$	first order + memory

- The step α_k comes from a **line search** on $\varphi(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{d}_k)$ (Ch. 2). “Exact” = minimize φ ; “inexact” = decrease enough.
- Stopping: $\|\nabla f(\mathbf{x}_k)\| \leq \varepsilon_g$; $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| / \max(1, \|\mathbf{x}_k\|) \leq \varepsilon_x$; $k \geq k_{\max}$.
- Cost: n_g , n_H counted separately from n_f (which includes the line search); with numerical derivatives everything becomes n_f : $2n$ per gradient, $2n^2 + 1$ per Hessian with gradient.
- Under suitable assumptions and an appropriate line search, all three seek *stationary points*; the Hessian there, if nondegenerate, classifies the point (semidefinite: inconclusive).

Ready-made counters: fminunc and SciPy

In MATLAB, `fminunc` (quasi-Newton by default) returns `out.funcCount` and `out.iterations`; with `'SpecifyObjectiveGradient'` the gradient is analytical, otherwise it is by finite differences (the solver's forward gradients: $n + 1$; in this course we count central ones, $2n - 2.6$).

```
opts = optimoptions('fminunc', 'SpecifyObjectiveGradient', true);  
[x, fx, ~, out] = fminunc(@fg, x0, opts); % fg returns [f, g]  
out.funcCount, out.iterations
```

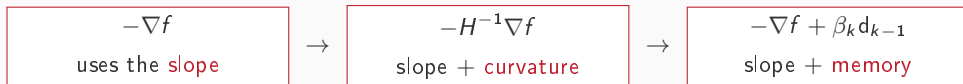
The full function (`fg`, which only computes the gradient when `nargout > 1`) is in the course companion code.

THE SAME IN PYTHON

```
from scipy.optimize import minimize  
r = minimize(f, x0, jac=grad, hess=hess,  
            method='Newton-CG')  
r.nfev, r.njev, r.nhev          # n_f, n_g, n_H  
# with 'CG'/'BFGS': drop hess= and r.nhev
```

The counters n_f , n_g , n_H come ready-made. The notebook `cap3_comparacao.ipynb` reproduces the comparison of the seven methods at the end of the chapter with the same rules (same problem, same x_0 , same criterion, cost in evaluations).

Three ways of choosing d_k



	information per iteration	cost with numerical derivatives (+ the line-search evaluations)
Steepest descent (3.3.1)	∇f	$2n$
Newton (3.3.2)	∇f and H	$2n^2 + 1$
Conjugate gradients (3.3.3)	∇f and d_{k-1}	$2n$

Let us start with the cheapest information: **if we only have the gradient, what is the natural direction?**

— 3.3.1, the steepest descent method.

References Full list: *Course bibliography*, at the end of the course PDF.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Chs. 2–3.
- Dennis, J. E., Jr., Schnabel, R. B., *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, SIAM (Classics in Applied Mathematics 16), 1996.

Resources

- Companion code: `fg` (f and gradient). MATLAB: `fminunc`, `optimoptions` (`'SpecifyObjectiveGradient'`), `out.funcCount`, `out.iterations`.
- Python: `scipy.optimize.minimize(..., method='Newton-CG')` (`r.nfev`, `r.njev`, `r.nhev`).
- Notebook `cap3_comparacao.ipynb`: comparison of the seven methods of the chapter.